Operator Overloading

Engr.Tehseen Ahsan

CS-102: Programming Fundamentals (PF)

CED 2020 Batch

Assistant Professor, Computer Engineering Department
HITEC University Taxila Cantt, Pakistan

1. Operator Overloading

- The process of defining additional meanings of operators is known as **operator overloading**.
- It enables an operator to perform different operations depending on the type of operands. It also enables the operators to process the user-define data types (i.e objects of a class).
- The basic arithmetic operators such as +, -, * and / normally work with basic data types such as **int, float** and **long** etc. However, an error will occur if these operators are used with user-defined objects.

1. Operator Overloading (Continue…)

Example:

- The addition operator is used to add two numeric values. Suppose a, b and c are three integer variables. The following statement will add the contents of **a** and **b** and store the result in variable **c**.

```
c = a + b;
```

- The addition operator already knows how to process integer operands. But it does not know how to process two user-defined objects. The above statement will generate error if **a**, **b** and **c** are objects of a class. However, operator overloading can enable the addition operator to manipulate two objects.

1.1. Overloading an Operator

An operator can be overloaded by declaring a special member function in the class. The member function uses the keyword **operator** with the symbol of operator to be overloaded.

Syntax- The syntax of overloading an operator is as follows:

```
return_type operator op ()
{
    function body;
};
```

Example:

```
void operator ++()
      {
function body;
}
```

1.2. Overloading Unary Operators

A type of operator that works with single operand is called **unary operator**. The unary operators are overloaded to increase their capabilities. The commonly used unary operators that can be overloaded in C++ are as follows:

Unary Operators	Description
!	Logical NOT
&	Address-of
~	One's complement
*	Pointer dereference
+	Unary plus
-	Unary negation
++	Increment
- -	Decrement

1.2.1. Overloading ++ Operator

The increment operator ++ is a unary operator. It works with single operand. It increases the value of operand by 1. It only works with numerical values by default. It can be overloaded to enable it to increase the values of data members of an object in the same way.

1.2.1. Overloading ++ Operator (continue…)

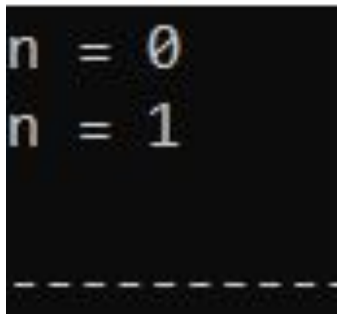
Program 1: Write a program that overloads increment operator to work with user-defined objects.

```
#include<iostream>
using namespace std;
class Count
{
private:
int n;
public:
        Count()   // Constructor Member Function
        {
                n = 0;
        }
        void show()   // Simple Member Function
        {
                cout<<"n = "<<n<<endl;
        }
        void operator ++()   // Operator overloaded Member Function
        {
                ++n;

        }
};
```

1.2.1. Overloading ++ Operator (continue…)

```
int main()
{
        Count obj;
        obj.show();
        ++obj;  // calling operator overloaded function
        obj.show();
        return 0;
}
```

Output



Different ways of implementing increment operator: ++n **or** n=n+1 **or** n+=1.

1.2.2. Operator Overloading with Returned Value

The increment operator can be used in assignment statement to store the incremented value in another variable. Suppose a and b are two integers. The following statement will increment the value of **a** by 1 and then assign the new value(incremented value) to **b**.

b = ++a; *//where a and b both must be objects of the same class.*

The syntax of special member function to be used for this purpose will look like:

```
class_type operator op()
{
function body;
return temp;
}
```

temp: *temporary object of the class created inside this function to store the incremented value.*

1.2.2. Operator Overloading with Returned Value (continue…)

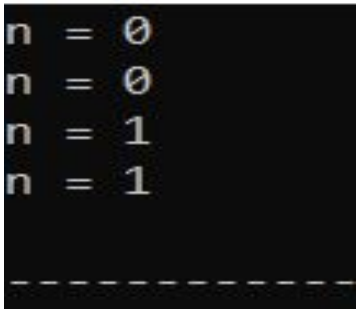
Program 2:

```
#include<iostream>
using namespace std;
class Count
{
private:
int n;
public:
        Count()   // Constructor Member Function
        {
                n = 0;
        }
        void show()   // Simple Member Function
        {
                cout<<"n = "<<n<<endl;
        }
        Count operator ++()   // Operator overloaded Member Function with return value
        {
                Count temp;
                        n = n+1;
                        temp.n = n;
                        return temp;
        }
};
```

1.2.2. Operator Overloading with Returned Value (continue…)

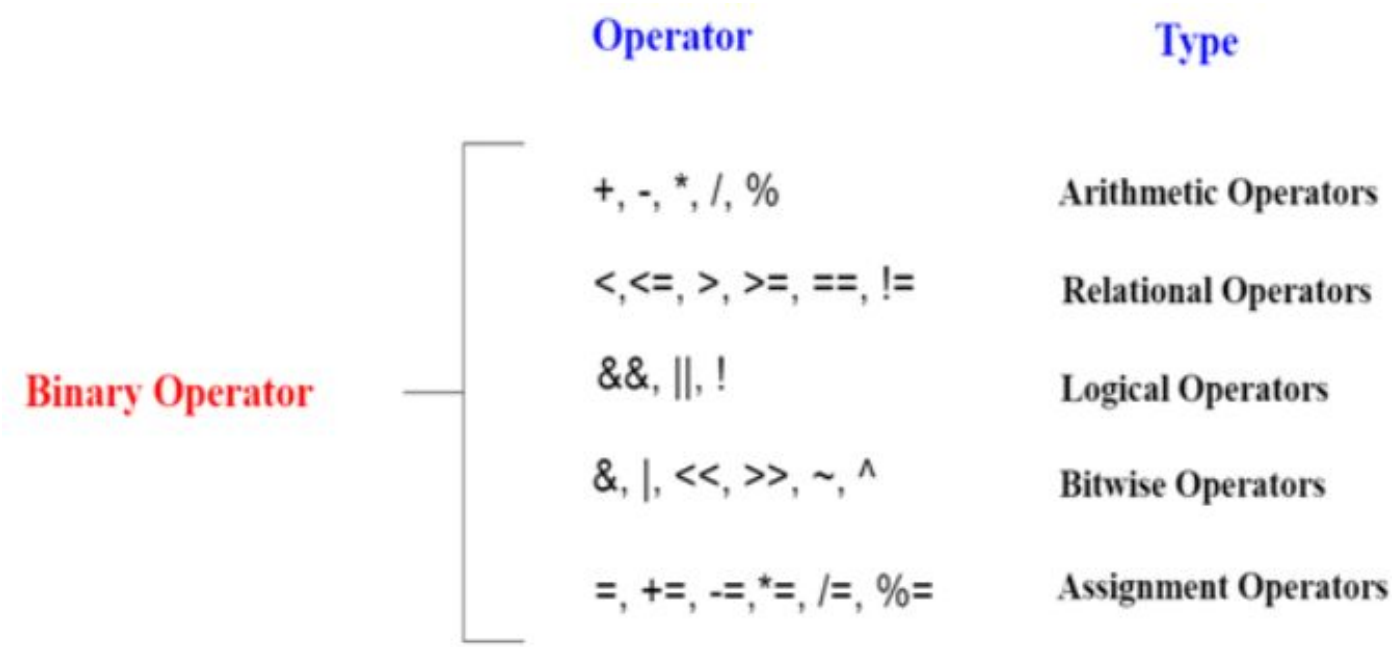
```
int main()
{
        Count x, y;
        x.show();
        y.show();
        y = ++x;
        x.show();
        y.show();
        return 0;}
```

Output



1.3. Overloading Binary Operators

A type of operator that works with two operands is called **binary operator**. The binary operators are overloaded to increase their capabilities. The commonly used binary operators that can be overloaded in C++ are as follows:



1.3. Overloading Binary Operators (continue…)

Program 3: Write a program that overloads binary addition operator +.

```
#include<iostream>
using namespace std;
class Add
{
private:
int a, b;
public:
Add()
        {
                a=b=0;
        }
        void in()
        {
                cout<<"Enter a: ";
                cin>>a;
                cout<<"Enter b: ";
                cin>>b;
        }
        void show()
        {
                cout<<"a = "<<a<<endl;
                cout<<"b = "<<b<<endl;}
```

1.3. Overloading Binary Operators (continue…)

```
Add operator +(Add p)
        {
                Add temp;
                temp.a = p.a + a;
                temp.b = p.b + b;
                return temp;
        }

};
int main()
{
        Add x, y, z;
        x.in();
        y.in();
        z = x + y;
        x.show();
        y.show();
        z.show();
        return 0;
}
```

Output

```
Enter a: 10
Enter b: 20
Enter a: 5
Enter b: 3
a = 10
b = 20
a = 5
b = 3
a = 15
b = 23
```

1.3. Overloading Binary Operators (continue…)

How Program 3 Works?

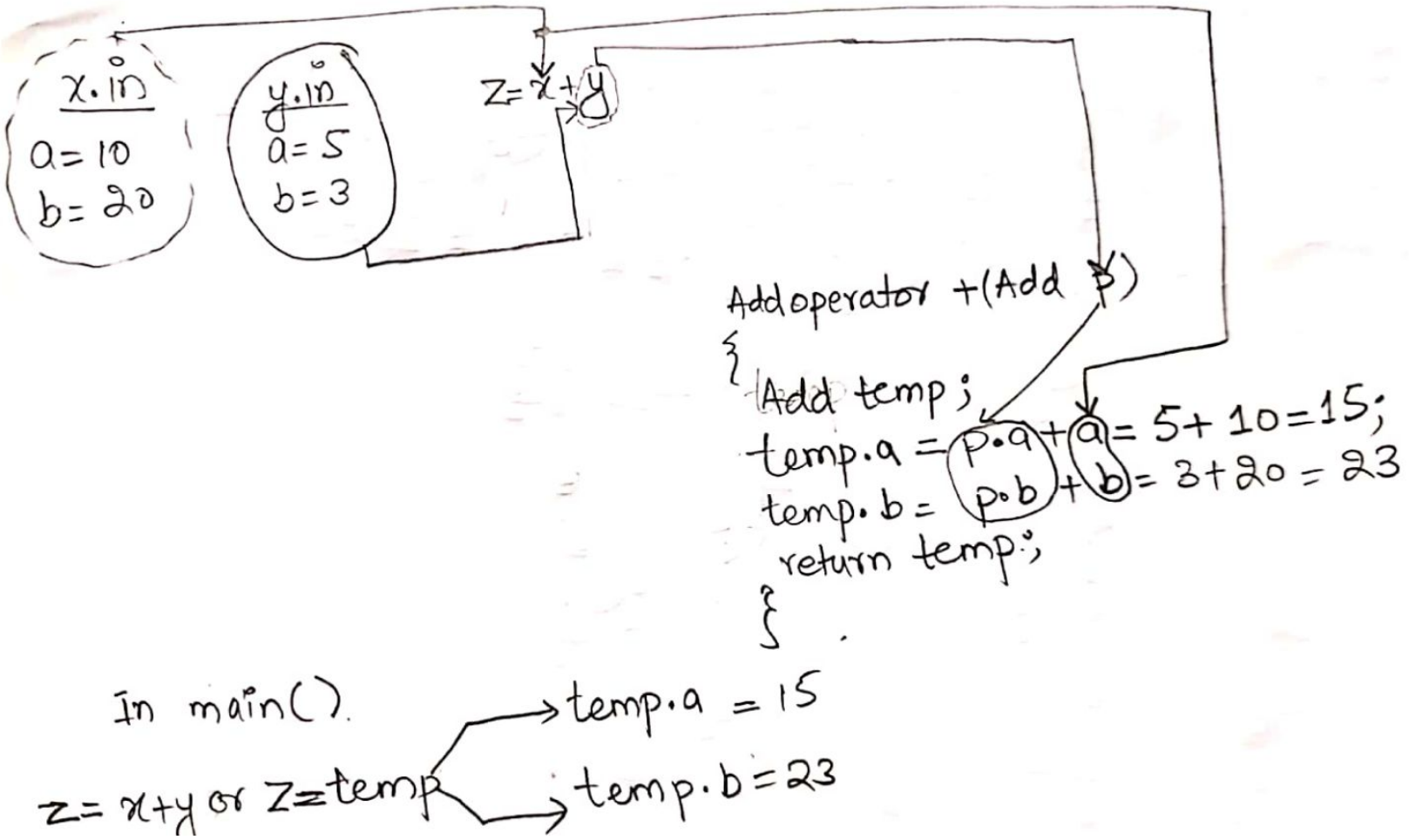
The program overloads the binary addition operator. *When the binary operator is used with objects, the left operand acts as calling object and the right operand acts as parameter passed to the member function*.

```
z = x + y;
```

In the above statement, **x** is the calling object and **y** is passed as parameter. The member function adds the values of calling object and parameter object. It stores the result in a temporary object (**temp**) and then returns the temporary object. The returned object is copied to the object on the right side (object **z** in this case) of assignment operator in **main()** function.

1.3. Overloading Binary Operators (continue…)

Program 3 Work flow:



1.3. Overloading Binary Operators (continue…)

Program 4: Write a program that overloads arithmetic addition operator + for concatenating two string values.

```
#include<iostream>
#include<string.h>
using namespace std;
class String
{
private:
char str[20];
public:
        String()
        {
                str[0] = '\0';
        }
        void in()
        {
                cout<<"Enter String: ";
                cin>>str;
        }
        void show()
        {
                cout<<str<<endl;
        }
```

1.3. Overloading Binary Operators (continue…)

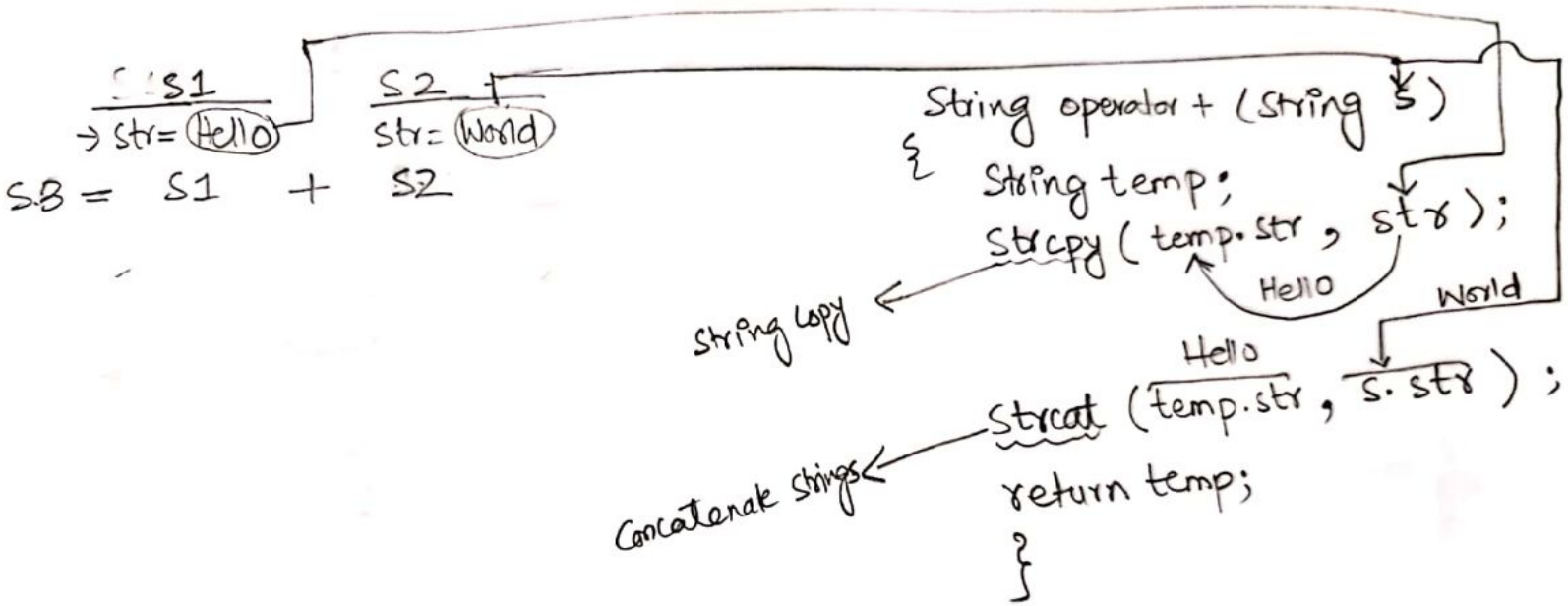
```
        String operator +(String s)
        {
                String temp;
                strcpy(temp.str, str); // copy contents of string str (Hello) into string temp.str.
                strcat(temp.str, s.str); // append copy of s.str at the end of temp.str
                return temp;
        }

};
int main()
{
        String s1, s2, s3;
        s1.in();
        s2.in();
        cout<<"s1 =";
        s1.show();
        cout<<"s2 =";
        s2.show();
        cout<<"Concatenating s1 and s2 in s3...."<<endl;
        s3 = s1 + s2;
        cout<<"s3 = ";
        s3.show();
        return 0;}
```

Output

```
Enter String: Hello
Enter String: World
s1 =Hello
s2 =World
Concatenating s1 and s2 in s3....
s3 = HelloWorld
```

1.3. Overloading Binary Operators (continue…)

Program 4 Work flow:



S3 = S1 + S2
or
S3 = temp
or
S3 = Hello World

References

1. *"C++ Programming: From Problem Analysis to Program Design",6/ed by D.S. Malik, (2013), CENGAGE Learning.*
2. *"Object Oriented Programming in C++," 4/ed by Robert Lafore (2001) .*
3. *"C++ How to Program," 5/ed by Deitel & Deitel (2005).*
4. *"Program with C++ Object Oriented Programming Aikman Series", by C.M Aslam and T.A Qureshi.*
5. *Related documents from open source, mainly internet.*